

# UART PROTOCOL

## Implementation, Simulation & Comparative Analysis

*A Study of UART Protocol Across Keil (8051 Assembly), Proteus Simulation, and Arduino (Tinkercad)*

Prepared by

**Mallampally Jayantha Siva Srinivas**

B.Tech, Electronics and Communication Engineering

## 1. Abstract

---

This project explores the implementation and verification of UART (Universal Asynchronous Receiver Transmitter) communication using three distinct development environments: Keil  $\mu$ Vision with 8051 Assembly language, Proteus Design Suite for circuit-level simulation, and Tinkercad Circuits using the Arduino platform. The objective is to demonstrate UART functionality across a low-level register-based approach and a high-level library-driven approach, and to draw a clear comparison of how code complexity, development time, and hardware abstraction differ between the two methodologies.

## 2. Objectives

---

- To understand the working principle of UART communication at both hardware register level and software abstraction level.
- To implement UART transmission using 8051 Assembly language in Keil  $\mu$ Vision.
- To validate the implementation through circuit simulation in Proteus using the generated HEX file.
- To implement equivalent UART functionality using Arduino in Tinkercad with built-in serial libraries.
- To compare both approaches in terms of code complexity, development effort, and practical usability.

## 3. Introduction to UART

---

UART is a widely used serial communication protocol that enables asynchronous data exchange between devices using just two signal lines — transmit (TX) and receive (RX). It is commonly used in embedded systems for communication between microcontrollers, sensors, and computers. Implementing UART requires configuring the baud rate, data frame format, and transmit/receive control logic, either manually through hardware registers (as in 8051 Assembly) or through abstracted software libraries (as in Arduino).

## 4. Tools and Platforms Used

---

| Platform             | Role in Project                                           | Output Verified                             |
|----------------------|-----------------------------------------------------------|---------------------------------------------|
| Keil $\mu$ Vision    | Assembly-level UART code development & debugging for 8051 | Disassembly window, register states         |
| Proteus Design Suite | Virtual circuit simulation using compiled HEX file        | Virtual Terminal showing transmitted string |
| Tinkercad Circuits   | Arduino UNO based UART                                    | Serial Monitor showing                      |

| Platform | Role in Project          | Output Verified        |
|----------|--------------------------|------------------------|
|          | implementation using C++ | repeated string output |

Table 1: Summary of platforms used and their role in the project

## 5. Implementation Details

### 5.1 Keil µVision — 8051 Microcontroller (Assembly Language)

UART communication was first implemented using the 8051 microcontroller in Keil µVision, programmed entirely in Assembly language. This involved direct configuration of the SCON (Serial Control) register for UART mode, the TMOD and TH1 registers for Timer 1 based baud rate generation, and manual handling of the SBUF register along with the TI (Transmit Interrupt) flag to control character-by-character transmission. This low-level approach offers complete visibility into how UART hardware operates internally.

```

org 0000h
mov scon, #50h    ; UART Mode 1, 8-bit UART
mov tmod, #20h    ; Timer 1, Mode 2 (8-bit auto-reload)
mov th1, #0fdh    ; Baud rate 9600
setb tr1          ; Start Timer 1
mov dptr, #message ; Load message address
transmit:
  clr a
next_char:
  movc a, @a+dptr ; Load character from message
  jz transmit     ; Repeat on null character
  mov sbuf, a     ; Send data
wait_transmit:
  jnb ti, wait_transmit ; Wait for transmission to complete
  clr ti          ; Clear transmit flag
  inc dptr        ; Next character
  sjmp next_char  ; Repeat
message: db 'Embedded Systems & IOT', 00h
end

```

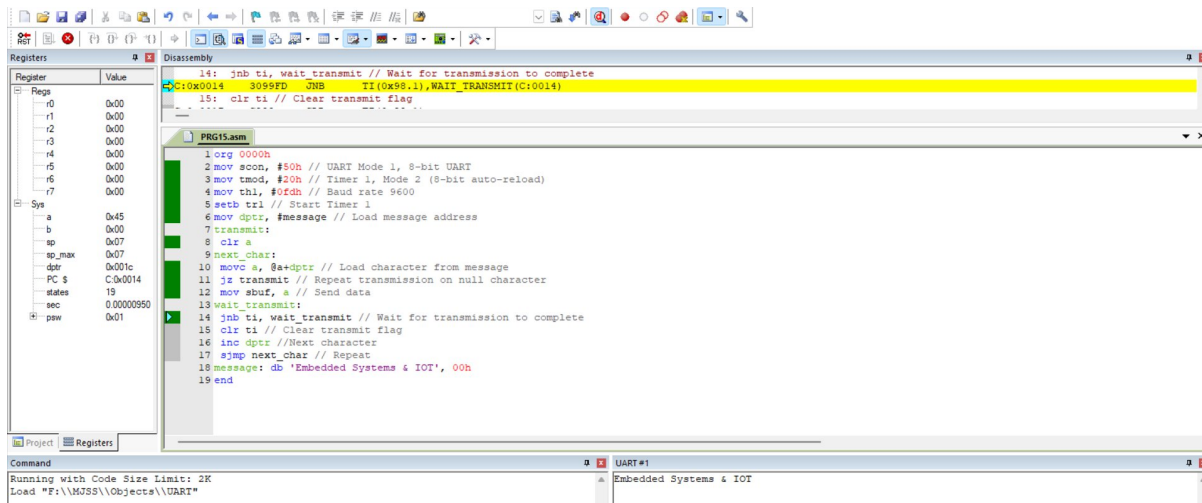


Figure 1: Keil µVision — Assembly code, disassembly view, and UART output ('Embedded Systems & IOT')

## 5.2 Proteus Design Suite — Circuit Simulation

The Assembly program developed in Keil was compiled into a HEX file and imported into Proteus Design Suite, where the 8051 (AT89C51) microcontroller circuit was simulated virtually. A Virtual Terminal was connected to the RXD/TXD pins to observe the transmitted data in real time. This step allowed verification of correct UART functionality without requiring physical hardware, confirming that the register-level configuration in the Assembly code produced the expected output.

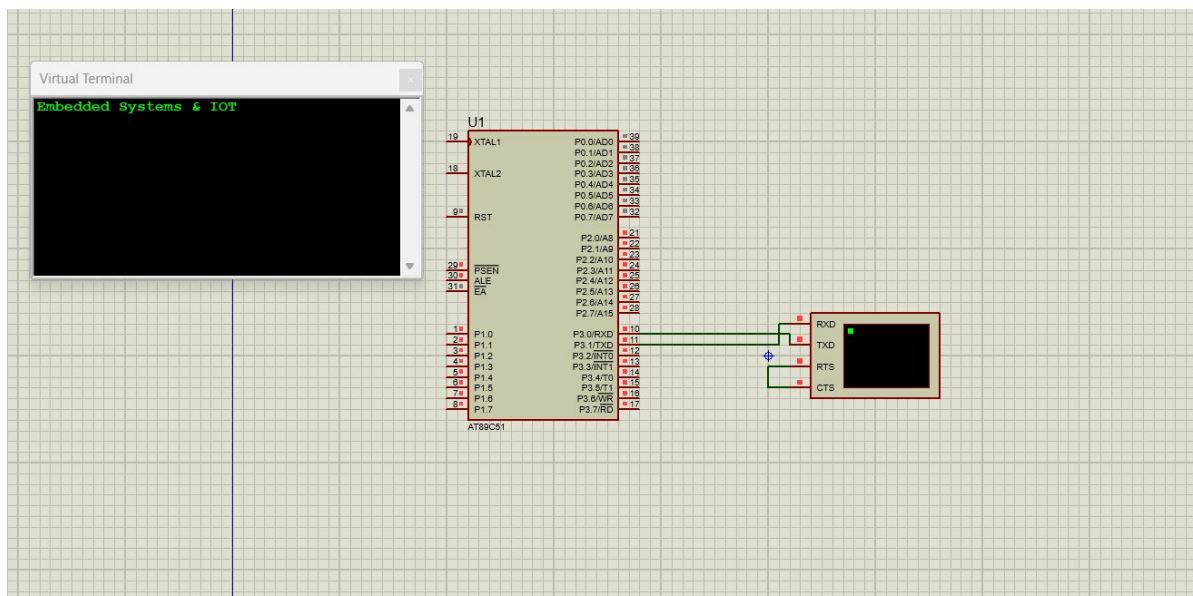


Figure 2: Proteus simulation — AT89C51 circuit with Virtual Terminal displaying received UART data

## 5.3 Tinkercad — Arduino UNO (C++ with Serial Library)

The same UART functionality was reimplemented using an Arduino UNO board in Tinkercad Circuits. Unlike the 8051 Assembly version, Arduino's built-in Serial library abstracts away all register-level configuration. A single `Serial.begin(9600)` call initializes UART communication, and `Serial.println()` handles framing and transmission of the data automatically, reducing the entire implementation to a handful of lines.

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.println("Embedded Systems & IOT");
  delay(1000);
}
```

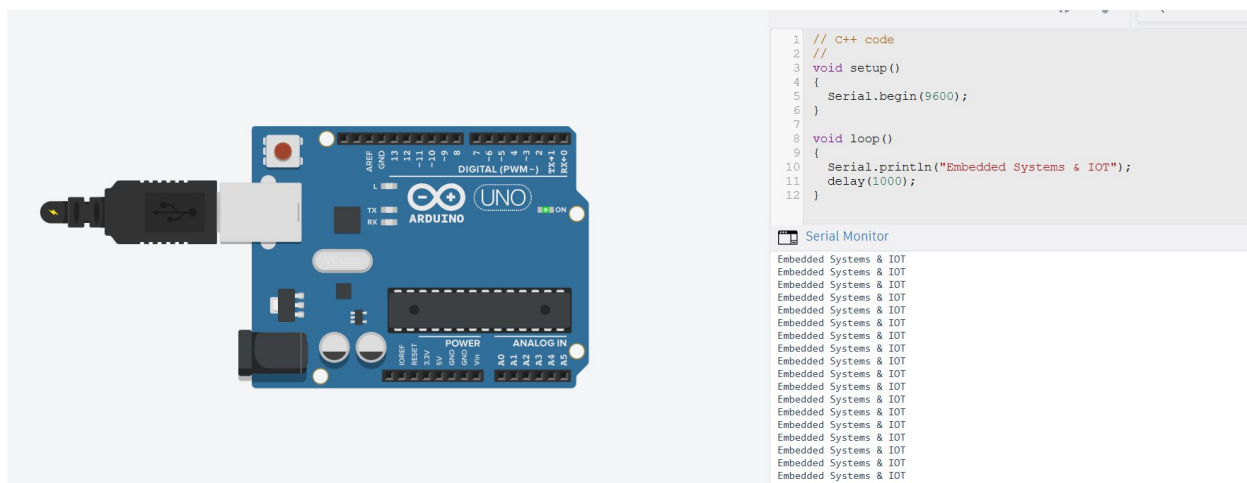


Figure 3: Tinkercad simulation — Arduino UNO with Serial Monitor output ('Embedded Systems & IOT')

## 6. Comparative Analysis: Assembly vs Arduino

The central comparison of this project lies in evaluating how the same UART functionality differs in complexity when implemented in 8051 Assembly versus Arduino C++. The table below breaks down this comparison across several practical dimensions.

| Aspect            | Keil / 8051 Assembly             | Arduino (Tinkercad)           |
|-------------------|----------------------------------|-------------------------------|
| Programming Level | Low-level, register-based        | High-level, library-based     |
| Lines of Code     | ~19 lines (manual control logic) | ~6 lines (built-in functions) |

| Aspect            | Keil / 8051 Assembly                         | Arduino (Tinkercad)                      |
|-------------------|----------------------------------------------|------------------------------------------|
| UART Setup        | Manual SCON, TMOD, TH1 register config       | Single call: Serial.begin(9600)          |
| Data Transmission | Manual SBUF load + TI flag polling loop      | Single call: Serial.println()            |
| Learning Curve    | Steep — requires hardware register knowledge | Gentle — abstracted hardware details     |
| Development Time  | Higher — more debugging at register level    | Lower — rapid prototyping                |
| Code Readability  | Moderate (needs inline comments)             | High — self-explanatory function names   |
| Hardware Control  | Full, precise control over UART hardware     | Limited to library-exposed functionality |
| Best Suited For   | Embedded systems education, optimization     | Rapid prototyping, IoT applications      |

Table 2: Code and development complexity comparison — 8051 Assembly vs Arduino

## 6.1 Key Observations

- **Approximately 68% fewer lines of code:** The Arduino implementation required only about 6 lines to achieve the same UART output that took roughly 19 lines of register-level Assembly code.
- **Abstraction vs control trade-off:** Arduino's Serial library trades fine-grained hardware control for speed and simplicity, while Assembly trades development speed for precise control over every register and timing parameter.
- **Debugging effort:** Assembly debugging in Keil required monitoring register states (SCON, TI flag, SBUF) and disassembly windows, whereas Arduino debugging was limited to reading the Serial Monitor output directly.
- **Educational value:** The Assembly implementation provided deeper insight into how UART hardware actually functions at the register level, which the Arduino abstraction conceals.

## 7. Conclusion

This project successfully demonstrated UART communication across three complementary platforms. The 8051 Assembly implementation in Keil  $\mu$ Vision provided a foundational, register-level understanding of UART hardware operation, validated through Proteus circuit simulation. In contrast, the Arduino implementation in Tinkercad showcased how high-level libraries can drastically simplify UART communication, reducing both code volume and development time while sacrificing some degree of hardware-level control. Together, these implementations

highlight the trade-off between low-level hardware mastery and high-level development efficiency — a key consideration when choosing a platform for embedded systems and IoT projects.